PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Lab (UE23CS252A) — Lab 1 Deliverables

Problem Statement #02: Campus & Academic Operations — *Automated Rubric Assignment Evaluator*

1. Problem Context & Overview

Academic evaluators require an automated workflow to process batch student code submissions, execute syntax and unit test suites, run rubric-based scoring breakdowns, and assign peer reviews without manual distribution overhead. The system isolates untrusted code in containers, applies instructor rubrics summing to 100%, and aggregates reports into LMS gradebooks.

Stakeholders & Actors: (1) **Student:** Submits project zip archives, views rubric score breakdowns, and performs peer reviews. (2) **Faculty Evaluator:** Configures rubrics & test specs, triggers batch evaluations, inspects flagged similarity, and overrides/exports grades. (3) **Automated Grading Engine (System Actor):** Executes sandboxed test suites, enforces timeout/RAM limits, and calculates metric tallies.

2. Complete Requirements Table

2.1 Functional Requirements (FR-001 to FR-005)

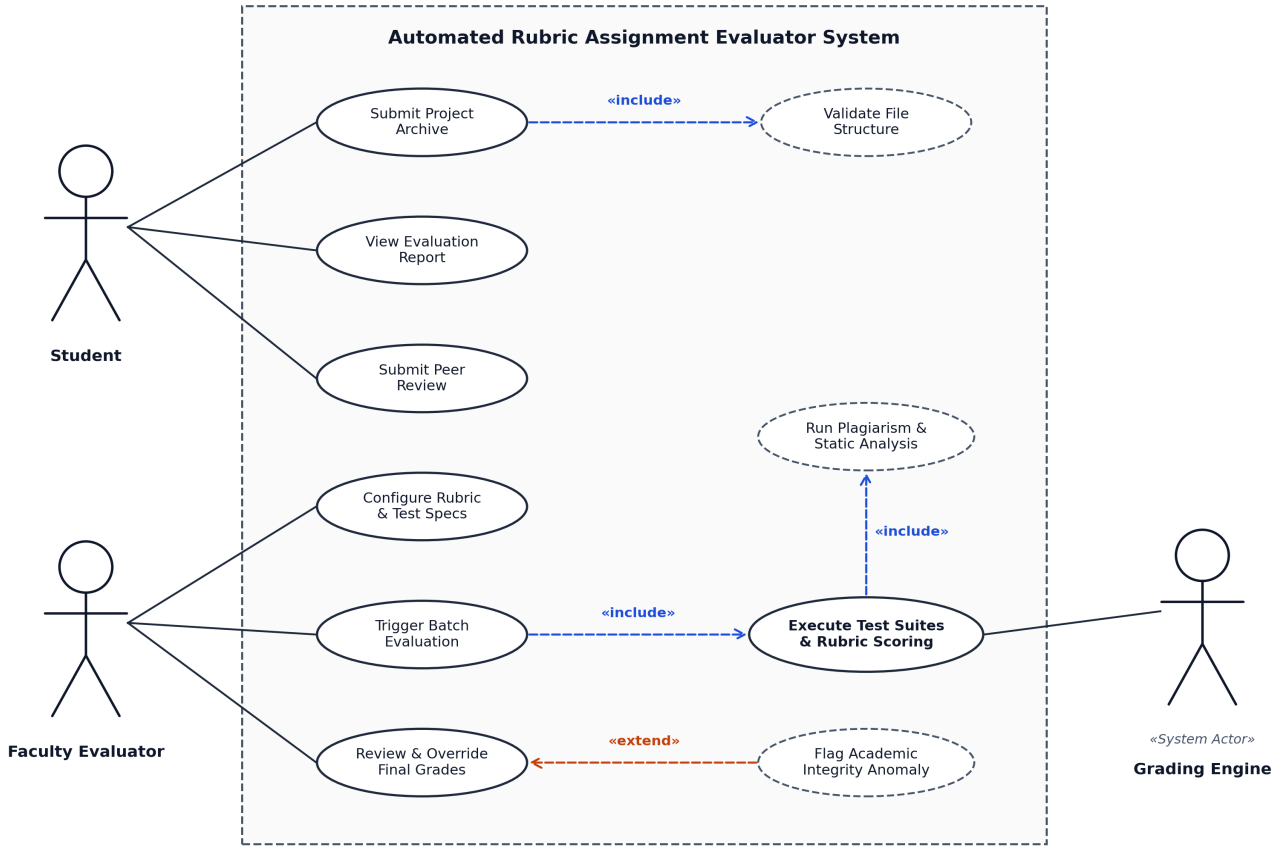
ID	Area / Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Automated Execution & Scoring	The system shall automatically ingest uploaded student project zip archives, queue them in isolated containers, execute pre-configured test suites, and generate an itemized rubric score breakdown.	**High**	**Pass:** Rubric rendered with test results within 10s of submission. **Fail:** Queue stalls, container crashes, or rubric tally is mathematically invalid.	Automates core grading mechanics and delivers instant, reproducible feedback.
FR-002	Rubric & Test Configuration	The system shall allow the Faculty Evaluator to create, update, and persist assignment rubrics with customizable criteria (unit tests, style, docs) and percentage weights summing to 100%.	**High**	**Pass:** System validates weights sum to 100% and applies configuration to next batch. **Fail:** Stale rubrics used or weights != 100% permitted.	Empowers faculty to tailor evaluation criteria to varying pedagogical goals.
FR-003	Plagiarism & Static Analysis	The system shall perform static syntax validation and source code similarity detection across batch submissions, flagging pairs exceeding a configurable similarity threshold (>30%).	**Medium**	**Pass:** Visual diff similarity report generated for flagged pairs without blocking batch runs. **Fail:** Plagiarism missed or crash on malformed code.	Upholds academic integrity and detects copied or uncompilable code automatically.
FR-004	Peer Review Allocation	The system shall automatically anonymize student submissions and distribute a configurable number of peer projects (e.g., 2-3 per student) with evaluation rubrics upon submission deadline.	**Medium**	**Pass:** Active students receive exact target count of distinct, anonymized peer submissions. **Fail:** Self-allocation, non-anonymized data, or duplicates.	Eliminates manual distribution overhead and facilitates collaborative peer learning.
FR-005	Grade Aggregation & Export	The system shall aggregate automated test scores, rubric metrics, and peer review scores into a consolidated gradebook, allowing Faculty to manually adjust marks, add remarks, and export to CSV/LMS format.	**High**	**Pass:** Faculty can view aggregated scores, override marks with audit remarks, and export CSV. **Fail:** Overrides lost or corrupted gradebook export.	Preserves faculty grading authority for edge cases and integrates with LMS systems.

2.2 Non-Functional Requirements (NFR-001 & NFR-002)

ID	Type / Attribute	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance & Scalability	The system shall handle up to 100 concurrent project submissions and test executions without dropping queue items or exceeding 1.5 GB memory footprint on the execution node.	**High**	**Pass:** Load tests with 100 simultaneous submissions maintain 100% completion with max RAM <= 1.5 GB and latency <= 15s. **Fail:** Dropped jobs, HTTP 504s, or OOM crashes.	Guarantees system responsiveness and zero data loss during high-stress deadline surges.
NFR-002	Security & Sandboxing	The system shall execute all untrusted student code inside restricted sandbox containers with isolated networking, restricted filesystem permissions, and strict execution timeouts (max 5s CPU time).	**High**	**Pass:** Malicious scripts (fork bombs, unauthorized disk access, socket binds) terminated immediately without host impact. **Fail:** Student code touches host filesystem or hangs workers indefinitely.	Protects institutional infrastructure from malicious attacks, exploits, and infinite loops.

3. UML Use-Case Diagram

The UML Use-Case Diagram below models all primary actors (Student, Faculty Evaluator, and Automated Grading Engine), system boundary, and core use cases, incorporating mandatory «include» and «extend» relationships.



Relationship Type	UML Dependency & Trigger Semantics
«include» (Mandatory)	• **Submit Project Archive** → **Validate File Structure:** Archive hierarchy and compression checks are unconditionally executed prior to queueing. • **Trigger Batch Evaluation** → **Execute Test Suites & Rubric Scoring:** Batch processing always executes test suite runners. • **Execute Test Suites** → **Run Plagiarism Check:** Static syntax and cohort code similarity are executed as part of scoring.
«extend» (Conditional)	• **Flag Plagiarism Anomaly** → **Review & Override Grades:** The system extends the instructor's grade review dashboard with an academic integrity alert *only when* pairwise source code similarity exceeds >30%.

4. Use-Case Flow Specification (Core 1-Page Document)

Use Case ID & Name	**UC-01: Process & Evaluate Assignment Submission**
Primary Actor	Student
Secondary Actors	Faculty Evaluator, Automated Grading Engine (Sandbox)
Description / Goal	Enables a student to upload a source code project archive (.zip), executes static checks and sandboxed unit tests, and generates an itemized rubric-based score breakdown report.
Trigger	Student clicks 'Submit for Evaluation' on the active assignment page.
Preconditions	1. Student is authenticated and enrolled in course. 2. Assignment window is open (current time < deadline). 3. Faculty Evaluator has published the active test suite and rubric.
Postconditions	**Success:** Archive securely stored, test suites executed, itemized rubric breakdown computed and rendered on dashboard, and gradebook updated. **Failure:** Submission rejected with diagnostic error logs (invalid format, compilation failure); 0 marks awarded for unexecuted criteria.

Main Success Scenario (MSS / Basic Flow):

1. Student navigates to the assignment submission portal and selects the target assignment.
2. Student uploads the project archive (.zip) and clicks 'Submit for Evaluation'.
3. System validates file format, size restrictions (≤50 MB), and folder structure (**«include» UC-02: Validate File Structure**).
4. System queues the submission into the automated evaluation worker queue.
5. Automated Grading Engine provisions an isolated container sandbox with restricted permissions.
6. Grading Engine compiles the code, executes unit tests against CPU/RAM limits, and checks code similarity (**«include» UC-04**).
7. System maps test outcomes to rubric criteria, tallies weighted scores, and generates an itemized score breakdown.
8. System commits submission metadata, test logs, and score tally to the persistent database.
9. System renders the itemized rubric breakdown to the Student and synchronizes the Faculty gradebook.

Alternate & Exception Flows:

Flow Identifier	Condition & Execution Path
Alternate Flow 1a: **Compilation / Syntax Error**	**At Step 6:** Grading Engine encounters compilation or syntax failure. **1a1.** Grading Engine halts test suite execution immediately. **1a2.** System allocates 0 marks to test-dependent rubric metrics and captures stderr logs. **1a3.** System marks status as 'Compilation Failed' and displays diagnostics to student. (Terminates at Postcondition).
Alternate Flow 2a: **Execution Timeout**	**At Step 6:** Student code exceeds the 5-second CPU limit. **2a1.** Sandbox watchdog terminates container; flags test case as 'Timeout Exceeded'. **2a2.** System tallies rubric score with penalty deductions and continues to Step 8.
Exception Flow 3a: **Invalid Archive Format**	**At Step 3:** Uploaded file is not a valid zip or violates directory hierarchy. **3a1.** System rejects file; alerts: *'Invalid archive format. Please check folder structure.'* **3a2.** Use case returns to Step 2 for re-upload.